

AGV Project Research — INFO1167 Lab #1 Robótica

1) Assignment deconstruction (from original PDF)

The lab asks for a simulation of **4 AGV robots** in a warehouse-like 2D map (QR nodes). Required behavior:

1. Robots start from base and are assigned carts.
2. Assignment is random (per simulation cycle).
3. Cart color reflects assigned robot.
4. Navigation is based on **graph algorithms**.
5. Robots carry carts to destination nodes.
6. Robots rotate in fixed angular steps (interpreted as $\sim 10^\circ$ from the PDF's "1G°" typo/context).
7. On delivery, robots release cart, return to base, recharge to 100%, and restart cycle.
8. Related theory: Python, Visual Python, geometry, DFS/shortest path/all paths.

2) Modeling choices

2.1 Warehouse as graph

- Node: `(x, y)` cell in a rectangular 2D grid.
- Edge: adjacency between horizontal/vertical neighbors.
- Cost: uniform (`1.0`) for shortest path demonstration.

Why this is correct:

- Matches "2D array where each cell is a QR node" in the lab text.
- Standard abstraction for mobile robot path planning in grid warehouses.

2.2 Robot finite-state machine

Implemented states:

- `IDLE_AT_BASE`
- `TO_CART`
- `TO_DESTINATION`
- `RETURN_TO_BASE`
- `CHARGING`

Why this is correct:

- Exactly mirrors the required operational cycle from the PDF.

2.3 Orientation and motion

- Robot heading uses geometry (`atan2`) to point to next node.
- Robot rotates by fixed increments (10°) before translation.
- If alignment is not enough, robot rotates this tick and moves next tick.

Why this is correct:

- Satisfies explicit rotation-step requirement.
- Adds physically plausible behavior compared to teleporting node transitions.

2.4 Routing algorithms

- **Operational routing**: Dijkstra shortest path.
- **Pedagogical evidence**: BFS and DFS helper methods included for report output.

Why this is correct:

- Dijkstra is robust and canonical for non-negative weighted edges.
- BFS/DFS coverage aligns with assignment themes (DFS, shortest path, route discovery).

3) Reliability and reproducibility design

- Fixed random seed support for deterministic re-runs.
- Explicit config dataclass for all simulation knobs.
- Tests validate:
 - path-finding correctness,
 - delivery eventually occurs,
 - state validity,
 - battery limits.

4) Reference notes used in investigation

1. ****Assignment-mandated graph reading****
 - Python graph essay: <https://www.python.org/doc/essays/graphs/>
2. ****Shortest path algorithm references****
 - NetworkX shortest path docs: https://networkx.org/documentation/stable/reference/algorithms/shortest_paths.html
 - Dijkstra overview: https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm
 - A* overview (for extension discussion): https://en.wikipedia.org/wiki/A*_search_algorithm
3. ****Multi-robot allocation perspective****
 - Gerkey & Mataric, MRTA framing via assignment problem: http://ai.stanford.edu/~gerkey/research/final_papers/icra03-oap.pdf

5) Scope implemented now vs extensions

Implemented now

- End-to-end simulation loop fully aligned with lab brief.
- Repeatable and testable architecture.
- Visual snapshot for presentation.

Natural next extensions

- Collision-avoidance constraints between robots.
- Time-window penalties and dynamic re-planning.
- A* + heuristic speedup on larger maps.
- Multi-objective assignment optimization (distance + battery + congestion).

6) Defense-ready explanation (short)

"We modeled the warehouse as a graph of QR nodes and implemented a finite-state AGV controller. Each robot is randomly assigned carts, plans shortest paths with Dijkstra, rotates in 10° increments before moving, delivers to target nodes, then returns to base and recharges. The cycle repeats automatically. We validated behavior through automated tests and deterministic seeded runs, and documented algorithmic alternatives (BFS/DFS/A*) for conceptual completeness."